

RESEARCH ARTICLE

Deep Reinforcement Learning-Based Pursuit–Evasion Strategy for USVs in Complex Multi-Obstacle Environments

JIAN LIU¹, XING SHEN², AND HONGWEI GAO^{1,2} ¹Naval Submarine Academy, Qingdao 266199, China²School of Mathematics and Statistics, Qingdao University, Qingdao 266100, China

Corresponding author: Hongwei Gao (gaohongwei@qdu.edu.cn)

This work was supported in part by the National Natural Science Foundation of China under Grant 72171126, and in part by the Systems Science Plus Joint Research Program of Qingdao University under Grant XT2024301.

ABSTRACT With the widespread application of Unmanned Surface Vehicles (USVs) in marine monitoring and military reconnaissance, collaborative pursuit in complex and dynamic environments faces significant challenges. Traditional path planning methods (e.g., A* algorithm, artificial potential field) suffer from limited adaptability to dynamic obstacles and difficulty in multi-objective collaborative optimization, while existing deep reinforcement learning approaches often neglect partial observability and variable-dimensional observations, resulting in insufficient strategy generalization. To address these challenges, this paper proposes a POSG-based deep reinforcement learning framework for multi-USV pursuit–evasion under partial observability, in which an enhanced Multi-Agent Soft Actor-Critic (MASAC) algorithm serves as the core learning component. By constructing hybrid observation spaces and a hierarchical reward function, the framework guides pursuer USVs to efficiently approach the target and form stable encirclement formations, while forcing the evader to adjust its escape trajectory in real time to delay capture. Through dynamic feature-compression encoding and obstacle-interaction pattern modeling, the proposed method alleviates the generalization challenge of policy networks in partially observable scenarios with varying input dimensions. Experimental results show that the proposed framework reaches stable convergence after approximately 3000 training episodes and achieves an 85% capture success rate over 100 evaluation episodes in unseen multi-obstacle maps, indicating improved capture efficiency and strong generalization under partial observability.

INDEX TERMS Collaborative target capture, multi-agent deep reinforcement learning, multi-agent soft actor-critic, pursuit-evasion game, unmanned surface vehicles.

I. INTRODUCTION

The pursuit–evasion game (PEG) is a classical problem formulation in differential game theory that has long served as a fundamental benchmark for cooperative control in multi-agent systems. It involves dynamic and adversarial interactions in which pursuers coordinate to capture targets, while evaders perform agile evasive maneuvers to escape. Unmanned Surface Vessels (USVs), owing to their high mobility and scalability, are well-suited for PEG tasks in

maritime environments. However, practical challenges—including underactuated kinematics, unknown and time-varying ocean disturbances, complex obstacle distributions, and limited onboard sensing—can substantially compromise the effectiveness of traditional methods.

To address these limitations, reinforcement learning (RL) has emerged as a promising data-driven approach for sequential decision-making under uncertainty. Unlike supervised learning, RL does not rely on labeled datasets but instead learns effective behaviors through interaction with the environment. The fusion of deep learning with RL has led to the advent of Deep Reinforcement Learn-

The associate editor coordinating the review of this manuscript and approving it for publication was Qiang Li .

ing (DRL), which enables scalable policy learning in complex, partially observable settings. Concurrently, Multi-Agent Reinforcement Learning (MARL) has emerged as a robust framework for modeling systems comprising multiple autonomous agents. By enabling agents to learn cooperative or competitive behaviors simultaneously, MARL facilitates more realistic and scalable solutions.

Despite these advancements, two critical limitations persist. First, most methods rely on the assumption of fully observable environments, thereby neglecting the partial observability constraints inherent to USVs' limited perception, which compromises policy robustness. Second, conventional MARL algorithms typically utilize fixed-dimensional inputs and exhibit tightly coupled decision-making between pursuit and obstacle avoidance. This architectural rigidity impedes scalability and adaptability in dynamic environments characterized by varying swarm sizes and obstacle distributions.

To address these issues, this study proposes a POSG-based deep reinforcement learning framework for multi-USV cooperative pursuit–evasion under partial observability. Specifically, the proposed “novel DRL-based approach” refers to the overall POSG modeling and learning framework, while the proposed enhanced MASAC is the core algorithmic instantiation used to solve the resulting decision-making problem. We model the problem as a POSG and develop an enhanced MASAC algorithm that integrates several key innovations:

To address the challenges of collaborative target capture under partial observability and dynamic adversarial interactions in multi-USV containment tasks, this study formulates the problem as a POSG and develops an enhanced MASAC algorithm. We devise a role-specific policy optimization framework: pursuers utilize a shared-weight encoder to bolster swarm formation coordination, whereas evaders employ an independent adversarial encoder to enhance individual agility. This approach significantly improves convergence efficiency in continuous action spaces and policy robustness in complex, unconstrained scenarios.

We introduce a Feature Embedding Block (FEB) and an Obstacle Awareness Block (OAB) to encode variable-dimensional observations and model obstacle–agent interaction patterns. These modules improve policy generalization and scalability for multi-USV cooperative decision-making under partial observability and obstacle-rich environments. We further propose a role-aware heterogeneous value (critic) network for chasers and evaders. Using role-specific state representations and adversarial training, the critic improves Q-function approximation and stabilizes learning.

To address these issues, this study proposes a novel DRL-based approach to multi-USV cooperative pursuit–evasion under partial observability. We model the problem as a Partially Observable Stochastic Game (POSG) and develop an enhanced Multi-Agent Soft Actor-Critic (MASAC) algorithm. The main contributions are summarized as follows:

- 1) **POSG modeling and CTDE learning:** We formulate the multi-USV pursuit–evasion process in unknown multi-obstacle environments as a POSG under the centralized training with decentralized execution (CTDE) paradigm.
- 2) **Hybrid observations with dynamic compression:** We construct hybrid observations that combine fixed-length containment features, variable-length inter-agent relational features, and beam-based obstacle sensing, and map them into fixed-dimensional representations via the Feature Embedding Block (FEB) and Obstacle Awareness Block (OAB).
- 3) **Hierarchical rewards and temporal fusion:** We design hierarchical reward shaping and incorporate temporal feature fusion via a masked self-attention (MSA) module with interval-based temporal sampling (ITS), improving robustness under partial observability. Simulation results show stable convergence and strong generalization (85% success in 100 unseen-map tests).

A structured review of prior PEG and MARL studies is provided in Section II. The remainder of this paper is organized as follows: Section III introduces the fundamental concepts of multi-agent reinforcement learning and pursuit–evasion differential games, including USV kinematic models and the principles of the SAC algorithm. Section III-C elaborates on the problem formulation for pursuit–evasion games, defining a POSG framework and geometric conditions for successful containment. Section V presents the core design of the MASAC algorithm, encompassing the hybrid observation space, a hierarchical reward design, action-space decoupling schemes, and the network architectures of the FEB and OAB modules. Section VI details the simulation setup and validates the effectiveness of the proposed method through analyses of the training dynamics and comparisons with multiple baseline algorithms. Section VII concludes the paper and outlines directions for future work.

II. RELATED WORK

Early PEG solutions largely rest on analytical control theories. Isaacs [1] derived optimal control strategies using variational methods under assumptions of full-state information and (often) linear dynamics. These assumptions are rarely satisfied in real-world USV applications, where the dynamics are typically nonlinear, nonholonomic, and only partially observable. Subsequent research extended PEG to multi-agent settings [2] and utilized Hamilton–Jacobi–Isaacs (HJI) equations [3] as well as reachability analysis in three-dimensional spaces [4]. Nevertheless, these methods remain restricted to simplified scenarios, suffer from high computational costs, and require accurate environmental modeling, limiting their applicability in dynamic maritime environments. In parallel, performance-guaranteed and constraint-aware optimal control has been actively studied for multi-agent and USV systems. For example,

Cao et al. [5] investigated optimal fixed-time control for human-in-the-loop multi-agent systems with actuator faults, providing fault-tolerant coordination with fixed-time convergence. Moreover, Cao et al. [6] proposed a prescribed performance-based optimal formation control framework for USVs with position constraints and yaw-angle time-varying partial constraints, highlighting principled handling of safety/constraint requirements in maritime scenarios.

To address these limitations, reinforcement learning (RL) has emerged as a promising data-driven approach for sequential decision-making under uncertainty. Unlike supervised learning, RL does not rely on labeled datasets but instead learns effective behaviors through interaction with the environment. Several studies have applied RL to pursuit–evasion problems. For instance, Wang et al. [7] used Q-learning combined with Apollonius-based geometric reasoning to derive (near-)Nash strategies. Fan et al. [8] proposed a deep RL framework incorporating noise-injected neural networks and prioritized replay to achieve real-time collision avoidance for USVs. However, these approaches often depend on handcrafted features and may not scale well in high-dimensional or unstructured environments.

The fusion of deep learning with RL has led to the advent of Deep Reinforcement Learning (DRL), which enables scalable policy learning in complex, partially observable settings. Wang et al. [9] leveraged a Deep Deterministic Policy Gradient (DDPG) approach for USV containment, achieving enhanced stability in evasion strategies. Concurrently, MARL has emerged as a robust framework for modeling systems comprising multiple autonomous agents. By enabling agents to learn cooperative or competitive behaviors simultaneously, MARL facilitates more realistic and scalable solutions.

Concurrently, substantial progress has been achieved in integrating deep learning with MARL to tackle complex multi-agent decision-making challenges. Specifically, algorithmic innovations such as value decomposition networks, actor–critic architectures tailored for cooperative settings, and credit assignment mechanisms have significantly enhanced performance in high-dimensional, uncertain environments [10], [11], [12]. These advancements lay a strong theoretical and algorithmic groundwork for applying MARL to USV-based pursuit–evasion scenarios.

Recent advancements in MARL have demonstrated significant potential for USV applications. Zhang et al. [13] developed a MADDPG-based multi-UAV pursuit strategy that emulates human prediction via target estimation networks. Li et al. [14] proposed a minimax-enhanced MADDPG algorithm to enhance generalization against adaptive adversaries. Focusing on maritime contexts, Qu et al. [15] presented a DRL-based PEG framework that integrates PPO with imitation learning for collaborative pursuit and evasion in multi-obstacle environments. Furthermore, several recent studies have tackled related challenges in USV navigation and coordination by leveraging RL techniques. For instance, a DRL-VO-based method was proposed for

human-like obstacle avoidance [16], while a double DQN-based planner was developed for global path planning in amphibious USVs [17]. Moreover, a distributed PEG strategy for perception-limited USV swarms was implemented via multi-agent PPO [18]. These contributions underscore the increasing prominence of DRL in maritime autonomous control.

III. PRELIMINARIES

A. USV MOTION MODEL

This paper focuses on the kinematic modeling of USVs in two-dimensional planar motion, employing a dual-coordinate system representation. The global motion is described using a geodetic coordinate system $X_E O_E Y_E$, while the local motion adopts a hull coordinate system $X_B O_B Y_B$, to establish a 3 degrees of freedom (DOF) motion model for USVs, as shown in Fig. 1. The origin O_B of the hull coordinate system is typically positioned at the USV's center of gravity, with axes oriented according to nautical standards: the X_B -axis aligns with the hull's longitudinal axis pointing forward, and the Y_B -axis extends perpendicularly to the portside. This coordinate system is particularly suitable for characterizing the directional attributes of the hull's velocity vectors. The symbolic definitions and physical interpretations of axial motion parameters are detailed in Table 1.

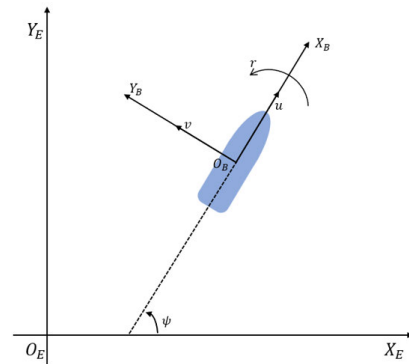


FIGURE 1. The schematic diagram of the 3 DOF coordinate system of a single USV.

TABLE 1. Coordinate variables of USV.

Term	Direction	Linear Velocity
surge	Along X_B -axis	u (m/s)
sway	Along Y_B -axis	v (m/s)
yaw	Counterclockwise rotation	r (rad/s)

For the 3 DOF kinematic modeling of USVs in planar motion, we define the position vector $\eta = [x, y, \psi]^T$ and the body-frame velocity vector $v = [u, v, r]^T$, where u and v denote surge and sway velocities, and r denotes yaw rate. The USV kinematic model is:

$$\dot{\eta} = R(\psi)v, \quad (1)$$

where $R(\psi)$ is the rotation matrix transforming the velocity vector from the body-fixed frame to the inertial frame. Its specific formulation is given by:

$$R(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2)$$

Simultaneously, considering the underactuated nature of USVs (typically incapable of generating propulsion along the hull's y-direction), the primary adjustable parameters are the x-direction linear velocity u and yaw angular velocity r . Consequently, the kinematic model of USVs can be simplified as:

$$\begin{cases} \dot{x}(t) = u(t) \cdot \cos \psi(t), \\ \dot{y}(t) = u(t) \cdot \sin \psi(t), \\ \dot{\psi}(t) = r(t). \end{cases} \quad (3)$$

B. ON-BOARD SENSORS

Where onboard sensors play a critical role in enabling autonomous navigation and pursuit-evasion tasks for USVs, the vehicle is equipped with a satellite communication and positioning system for real-time acquisition of its own and swarm members' position coordinates (x_i, y_i) , as well as an attitude sensor, gyroscope, and inertial navigation system (INS) to obtain real-time yaw angular ψ , linear velocity (u, v) , and angular velocity r . Additionally, the USV incorporates a radar system for direct positioning of adversarial USVs and a forward-looking sonar detection system with beam sensors for environmental obstacle perception, as shown in Fig. 2.

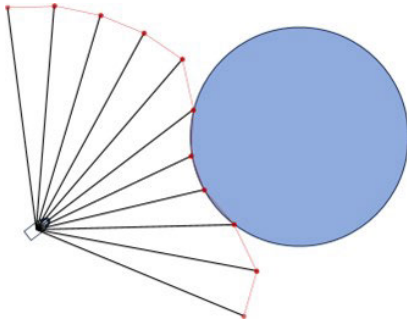


FIGURE 2. Ray sensor schematic.

Where the forward-looking sonar detection module features a sector-shaped scanning area with an angular aperture of 120° , configured with 11 detection beams, each having a maximum scanning range of 20 meters. When a beam detects an obstacle, the linear distance from the USV's position (sector center) to the obstacle contact point is calculated. If no obstacle is detected along a beam, the module returns the predefined maximum detection distance. The final output of the detection module is a distance vector composed of individual beam detection results, formally expressed as: $D(t) = [d_1(t), d_2(t), \dots, d_{11}(t)]$.

C. SAC ALGORITHM

The Soft Actor-Critic (SAC) algorithm is a policy optimization method based on maximum entropy reinforcement learning, initially proposed by Haarnoja et al. in 2018 [19]. As an off-policy Actor-Critic approach integrated with the maximum entropy reinforcement learning framework, SAC extends the conventional Actor-Critic architecture by introducing a policy entropy regularization mechanism. This formulation simultaneously optimizes cumulative rewards and maximizes policy stochasticity, effectively balancing the exploration-exploitation trade-off.

To stabilize the learning process, the Critic employs a dual-network architecture consisting of a Q-network Q_ω and a target Q-network $Q_{\bar{\omega}}$. The objective is to minimize the Bellman residual. We first define the target value y_t , which incorporates the entropy-augmented reward to encourage exploration:

$$y_t = r_t + \gamma (Q_{\bar{\omega}}(s_{t+1}, a_{t+1}) + \alpha H(\pi(\cdot|s_{t+1}))), \quad (4)$$

where $a_{t+1} \sim \pi_\theta(\cdot|s_{t+1})$ is the next action sampled from the current policy. The entropy term $H(\pi(\cdot|s_{t+1})) = \mathbb{E}_a[-\log \pi_\theta(a|s_{t+1})]$ measures the policy's stochasticity, and α is the temperature parameter balancing reward maximization and entropy. Consequently, the loss function for the Q-network is defined as the expected mean squared error between the estimated Q-value and the target value:

$$L_Q(\omega) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} \left[\frac{1}{2} (Q_\omega(s_t, a_t) - y_t)^2 \right], \quad (5)$$

where $\mathcal{D}$ denotes the replay buffer. The parameters ω are updated via gradient descent, while the target parameters $\bar{\omega}$ track ω using a soft update mechanism.

$$\begin{aligned} \nabla_\omega L_Q(\omega) &= \nabla_\omega Q_\omega(s_t, a_t) (Q_\omega(s_t, a_t) \\ &\quad - (r_t + \gamma (Q_{\bar{\omega}}(s_{t+1}, a_{t+1}) - \alpha \log(\pi_\theta(a_{t+1}|s_{t+1}))))). \end{aligned} \quad (6)$$

The Actor network optimizes the policy parameters θ by minimizing the expected KL-divergence between the policy and the Boltzmann distribution induced by the Q-function. Specifically, the loss function is formulated as:

$$L_\pi(\theta) = \mathbb{E}_{s_t \sim \mathcal{D}, a_t \sim \pi_\theta} [\alpha \log \pi_\theta(a_t|s_t) - Q_\omega(s_t, a_t)]. \quad (7)$$

where $\mathcal{D}$ denotes the replay buffer, and a_t is sampled from the current policy given state s_t .

In continuous action spaces, the policy outputs the parameters of a Gaussian distribution. Since the sampling operation blocks gradient propagation, we employ the reparameterization trick to allow gradients to backpropagate through the action sampling process:

$$a_t = f_\theta(\epsilon_t; s_t), \quad (8)$$

where ϵ_t is a noise vector sampled from a fixed distribution (e.g., spherical Gaussian $\mathcal{N}(\mathbf{0}, \mathbf{I})$). By substituting Eq. (8)

into Eq. (7), the objective is reformulated as an expectation over the state and noise:

$$L_{\pi}(\theta) = E_{s_t, \delta_t} [\alpha \log(\pi_{\theta}(f_{\theta}(s_t) | s_t)) - Q_{\omega}(s_t, f_{\theta}(s_t))], \quad (9)$$

where the policy distribution π_{θ} is implicitly defined by the transformation f_{θ} . This formulation enables the gradient of Eq. (9) to be estimated via the chain rule as follows:

$$\begin{aligned} \nabla_{\theta} L_{\pi}(\theta) &= \nabla_{\theta} \alpha \log(\pi_{\theta}(a_t | s_t)) \\ &+ (\nabla_{a_t} \alpha \log(\pi_{\theta}(a_t | s_t)) - \nabla_{a_t} Q(s_t, a_t)) \nabla_{\theta} f_{\theta}(\delta_t; s_t). \end{aligned} \quad (10)$$

IV. PROBLEM FORMULATION AND MODELING

A. DESCRIPTION OF PURSUIT-ESCAPE GAME PROBLEMS

For the multi-agent collaborative target capture problem in unknown environments, this study addresses scenarios with unknown target trajectories and obstacle interference. The pursuer swarm dynamically predicts target escape paths and optimizes containment trajectories and formation patterns in real-time, leveraging local perceptual data and collaborative communication mechanisms. By establishing a pursuit-evasion differential game framework, the system achieves multi-agent coordinated blockade and progressive compression of the target's operational domain, while satisfying USV kinematic constraints, environmental boundary limitations, and collision avoidance requirements. The objective is to effectively complete the containment operation against the evader USV within specified time thresholds. Simultaneously, the evader USV continuously optimizes its escape trajectory based on the pursuers' containment paths and formation configurations to evade encirclement or maximally delay capture.

Underlying mechanism (how the proposed method addresses the concerned problem): The core difficulty of the considered task is not merely to “reach” the evader, but to enforce a *geometric closure* under partial observability and unknown obstacle interference. To make this mechanism explicit, we model the interaction as a multi-pursuer single-evader *zero-sum differential game* and implement it as a *Partially Observable Stochastic Game (POSG)*. Mechanistically, (i) the differential-game formulation couples the pursuers' objective (containment) and the evader's objective (escape) into a single adversarial criterion, which prevents the learned policy from exploiting non-adversarial behaviors; (ii) the POSG observation design encodes the minimal local geometric information needed for coordinated encirclement (relative bearings, relative distances, adjacency-defined capture angles, and obstacle proximity from beam sensors), thus turning the global closure requirement into locally computable signals; and (iii) the hierarchical reward decomposes containment into complementary sub-mechanisms—fast approach, arrival synchronization, collision avoidance, and phase-balanced

angle regulation—so that the emergent team behavior becomes “blockade-and-compress” rather than greedy interception.

Successful capture is defined by the following criteria: (1) The distance between any pursuer USV and the evader USV must be less than the capture radius R_{cap} ; (2) The capture angle between adjacent pursuer USVs shall not exceed π . A specific capture scenario meeting these geometric constraints is shown in Fig. 3, where USVa, USVb, USVc denotes pursuer USVs, USV_T represents the evader USV, R_{cap} specifies the capture radius, θ_{ab} and θ_{ba} indicate the relative heading angles between USVa and USVb, θ_{aT} is the relative heading angle of USVa and USV_T, γ_{ab} defines the capture angle between USVa and USVb, d_{ab} and d_{aT} measure the distances between USVa and USVb or USVa and USV_T, respectively.

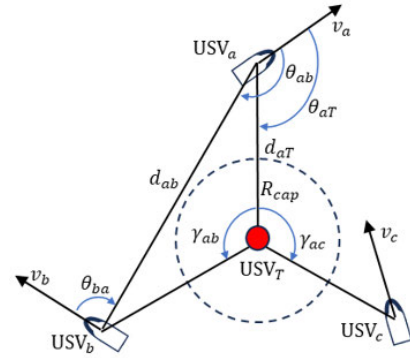


FIGURE 3. Schematic diagram of a three-USV capture scenario.

The pursuit-evasion game model involves two distinct roles: pursuers and an evader. The pursuers aim to capture the evader, while the evader seeks to evade the pursuers. This study formulates the target containment problem as a multi-pursuer-single-evader zero-sum differential game. According to the Nash equilibrium definition in game theory, the action strategies of both parties are optimal if and only if the following inequality holds:

$$J(\pi_p, \pi_e^*) \leq J(\pi_p^*, \pi_e^*) \leq J(\pi_p^*, \pi_e), \quad (11)$$

where J represents the pursuers' payoff function, π_p denotes the pursuers' Nash equilibrium strategy, and π_e signifies the evader's equilibrium strategy, (11) implies that if one party adopts the Nash equilibrium strategy while the other employs a non-rational strategy (i.e., deviating from the equilibrium), the non-compliant party cannot optimize its payoff function. The designed payoff functions are formulated as:

$$J(\pi_p, \pi_e) = \int_0^{t_f} \gamma^t R_p dt. \quad (12)$$

The payoff function incorporates two critical components: t_f denotes the time required for pursuer USVs to successfully contain the evader USV, while R_p represents the designed reward for pursuer swarms based on their kinematic characteristics, which will be detailed in the subsequent section.

Within the pursuit-evasion differential game framework, pursuers are designed to maximize Equation 12, whereas the evader aims to minimize it.

B. PROBLEM MODELING

This paper models the pursuit-evasion game process of USVs as a Partially Observable Stochastic Game (POSG), denoted as $\langle N, S, O, A, P, \Phi, \{r_i\}_{i=1}^N, \gamma \rangle$, where N represents the total number of agents (including cooperative pursuers and adversarial evaders), S indicates the global state space of the environment, $O = O^1 \times \dots \times O^N$ corresponds to the observation space, O^i denotes the set of local observations receivable by agent i , $A = A^1 \times \dots \times A^N$ defines the joint action space, A^i specifies the executable action set for agent i , $P(s'|s, \mathbf{a})$ describes the state transition probability from state s to s' under joint action $\mathbf{a} = (a_1, \dots, a_N)$, $\Phi(o_1, \dots, o_N|s, \mathbf{a})$ is the observation function, characterizes the observation probability distribution conditioned on state s and joint action $\mathbf{a}$, $o_1, \dots, o_N$ represents the individual reward function for agent $\{r_i(s, \mathbf{a})\}_{i=1}^N$ dependent on state s and joint action $\mathbf{a}$, and $0 \leq \gamma \leq 1$ is the discount factor.

Mechanism-to-model connection: The POSG formalization is used because each USV makes decisions based on local sensing and limited communication; thus the global closure objective must be induced through (a) locally available geometric observations and (b) a reward that explicitly evaluates closure progress. Accordingly, we (1) define adjacency based on bearings to identify which capture angles each pursuer should regulate, (2) augment observations with beam-sensor obstacle proximity to prevent “closure” from being achieved via unsafe shortcuts, and (3) couple all agents through a zero-sum reward structure so that containment strategies remain robust against an adaptive evader.

The action space A should be designed for simplicity and efficiency, enhancing learning tractability while improving final algorithmic performance. For the USV pursuit-evasion task, we establish a shared continuous action space for both pursuers and evaders based on kinematic modeling. Considering the underactuated nature of USVs, the primary adjustable parameters are the surge-direction linear velocity u and yaw angular velocity r . Consequently, the action space is constructed as follows:

$$A_i = \{u_i, r_i\}, \quad (13)$$

where index $i \in \{p_1, p_2, \dots, p_N, e\}$ denotes N pursuer USVs and $i = e$ represents the evader USV, $x_j(t), y_j(t), \psi_j(t)$ define the coordinates and heading angle of the j -th USV at time t , $u_j(t), r_j(t)$ represent its surge velocity and yaw angular velocity, respectively. The surge velocity u_j and yaw angular velocity r_j of pursuer USVs are constrained by upper limits $u_j \in [0, u_{\max}^p]$, $r_j \in [-r_{\max}^p, r_{\max}^p]$, $j \in \{p_1, p_2, \dots, p_N\}$, while the evader USV's surge velocity u_e and yaw angular velocity r_e are bounded by $u_e \in [0, u_{\max}^e]$, $r_e \in [-r_{\max}^e, r_{\max}^e]$, where $u_{\max}^p, r_{\max}^p, u_{\max}^e, r_{\max}^e$ are constant parameters satisfying $u_{\max}^p > u_{\max}^e, r_{\max}^p < r_{\max}^e$.

The state space S encapsulates all perceptible information for agent decision-making and long-term reward evaluation. A well-designed state space ensures rapid algorithm convergence and enhances final performance. To address the distinct requirements of pursuit and evasion tasks, we propose the following specialized state spaces.

Two pursuer USVs are defined as adjacent if no other USVs exist between their bearing angles relative to the target. Fig. 4 illustrates the state features of USVa, where USVb and USVc are adjacent to USVa.

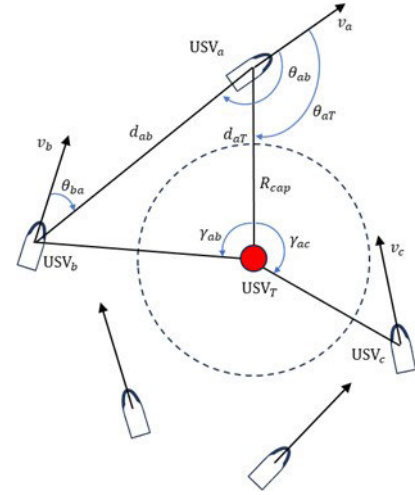


FIGURE 4. Schematic diagram of pursuit USV state characteristics.

For the a -th pursuer USV, we augment its observation space by incorporating state features that characterize containment angles alongside positional relationships. A beam sensor-enhanced observation space O_p^a is designed as follows:

$$O_p^a = \{O_{out}^a, O_{in}^a, D_a(t)\}. \quad (14)$$

The pursuer observation O_p^a in Eq. (14) is a *hybrid* observation, consisting of a fixed-size part and a variable-size part. Specifically, $O_{out}^a \in \mathbb{R}^3$, $D_a(t) \in \mathbb{R}^{11}$ (11 beam distances), and the internal observation contains $(N - 1)$ homogeneous inter-agent items with 5 attributes, which can be represented as $O_{in}^a \in \mathbb{R}^{(N-1) \times 5}$. To enable a single policy network to generalize to varying swarm sizes N , we map the hybrid observation into fixed-dimensional embeddings before feeding it into the policy/value networks: $z_a^{out} = f_{out}(O_{out}^a) \in \mathbb{R}^{32}$, $z_a^{var} = f_{var}(O_{in}^a) \in \mathbb{R}^{32}$, $z_a^{obs} = f_{obs}(D_a(t)) \in \mathbb{R}^{32}$, and concatenate them as $\hat{o}_a = [z_a^{out}; z_a^{var}; z_a^{obs}] \in \mathbb{R}^{96}$. The concrete implementations of $f_{var}(\cdot)$ and $f_{obs}(\cdot)$ are described later in Section IV.

Based on the containment success criteria and incorporating positional relationships and angular containment features, we construct the state space for pursuer USVs, with the containment scenario illustrated in Fig. 3. Then the observation space O_p^a of the USV for the a -th pursuer can be defined as the external observation space O_{out}^a , the internal observation space O_{in}^a , and the distance vector $D_a(t)$ of the ray sensor.

The external observation space O_{out}^a describes the relevant information of the evader USV perceived by pursuer USVa. It includes the relative heading angle θ_{aT} of USVa and USV_T, the remaining acquisition distance $d_a = d_{aT} - R_{cap}$, and the average value of the remaining acquisition distance $\bar{d}$.

The internal observation space O_{in}^a describes the characteristics of pursuer USVa and other USVs in the cluster. It includes the distance d_{ab} from other pursuer USVs and the relative heading angle θ_{ab}, θ_{ba} between them, the difference $\Delta d_{ab} = d_a - d_b$ of the remaining capture distance, and the capture angle γ_{ab} between pairwise pursuer USVs.

In a word, the external observation space O_{out}^a and the internal observation space O_{in}^a can be expressed by the following formula:

$$\begin{cases} O_{out}^a = \{\theta_{aT}, d_a, \bar{d}\}, \\ O_{in}^a = \{d_{ab}, \theta_{ab}, \theta_{ba}, \Delta d_{ab}, \gamma_{ab}\}, b \in N, b \neq a. \end{cases} \quad (15)$$

For the evader USV, an observation space O_e integrated with beam sensors is designed based on its task requirements, comprising the following components:

$$O_e = \{O_{out}^e, D_e(t)\}, \quad (16)$$

where $O_{out}^e = \{\phi_a, d_a\}$, $a \in N$ denotes the external observation features of the evader USV, including the relative heading angle ϕ_a between the evader USV and pursuer USVa, and the remaining capture distance d_a , while $D_e(t)$ represents the distance vector from the evader USV's beam sensor.

The evader observation O_e in Eq. (16) also contains a variable-size component. In particular, $O_{out}^e = \{\phi_a, d_a\}_{a \in N}$ scales with the number of pursuers and can be written as $O_{out}^e \in \mathbb{R}^{N \times 2}$, $D_e(t) \in \mathbb{R}^{11}$. For network input, we construct fixed-dimensional embeddings as $z_e^{var} = f_{var}(O_{out}^e) \in \mathbb{R}^{32}$, $z_e^{obs} = f_{obs}(D_e(t)) \in \mathbb{R}^{32}$, and use the concatenation $\hat{o}_e = [z_e^{var}; z_e^{obs}] \in \mathbb{R}^{64}$.

The perception capabilities of USVs are constrained by technological limitations in sensors and communication systems, preventing comprehensive environmental awareness and necessitating reliance on beam sensors for localized data acquisition. This partial observability inherently drives decision-making strategies toward probabilistic selection characteristics and real-time adaptive capabilities, thereby enabling more authentic replication of dynamic search environments encountered in practical scenarios.

Based on the different task requirements of chaser USV and evader USV in Section IV-A, the following reward function is designed to evaluate the current state of USVs.

For the pursuer USVs, consider the a -th pursuer as an example. Its overall reward is a weighted sum of four components: the target reward, collision reward, distance reward, and angle reward. Specifically,

$$R_p^a = \sigma_g r_{goal}^a + \sigma_{col} r_{collision}^a + \sigma_d r_{distance}^a + \sigma_a r_{angle}^a, \quad (17)$$

where $\sigma_g, \sigma_{col}, \sigma_d$ and σ_a denote the weighting coefficients of the corresponding reward terms.

The target reward represents r_{goal}^a that when the pursuer USV cluster completes the round up task, it should be given a larger reward $\delta_g > 0$,

$$r_{goal}^a = \begin{cases} \delta_g, & goal \\ 0, & otherwise \end{cases} \quad (18)$$

Collision reward $r_{collision}^a$ means that USVa collides with an obstacle or other USVs in the obstacle group, and a certain penalty shall be given $\delta_{col} > 0$,

$$r_{collision}^a = \begin{cases} \delta_{col}, & collision \\ 0, & otherwise \end{cases} \quad (19)$$

The distance reward $r_{distance}^a$ is to encourage the pursuit USVa to reach the elusive USV as soon as possible, and also hope that other pursuers USV will arrive at the same time as possible for joint capture. Therefore, the reward $r_{distance}^a$ is defined as follows:

$$r_{distance}^a = -\left(k_{d1}d_a + k_{d2}e^{(d_a - \bar{d})/\sigma(d)} - 1\right), \quad (20)$$

where k_{d1}, k_{d2} is the reward coefficient, $\bar{d}$ is the average value of the remaining capture distance, $\sigma(d) = \sum_{i=1}^N \sqrt{(d_i - \bar{d})^2} / N$. Through the dense reward composed of $r_{distance}^a$, when the USVa is far away from the evader USV, or when the distance between the USVa and the evader USV is different from other pursuer USV, a certain degree of punishment can be carried out to ensure that the pursuer USV cluster can reach the evader USV at the same time. To avoid numerical instability caused by extreme values of the exponential term, the normalized deviation $(d_a - \bar{d}) / \sigma(d)$ is clipped to a moderate range before exponentiation in implementation.

To achieve a stable encirclement with equal phase division during the pursuit, each pursuer USV is required to maintain, as closely as possible, a capture angle of $2\pi/N$ with its two adjacent pursuers. Let USV_b and USV_c denote the two pursuers adjacent to USV_a. The angle reward r_{angle}^a is defined as follows:

$$\begin{cases} r_{angle}^a = k_{a1}r_{a1} + k_{a2}r_{a2}, \\ r_{a1} = e^{-|\gamma_{ab} - 2\pi/N|} + e^{-|\gamma_{ac} - 2\pi/N|} - 2, \\ r_{a2} = e^{-\Delta\gamma_a} - 1, \end{cases} \quad (21)$$

where k_{a1} and k_{a2} are weighting coefficients. r_{a1} penalizes the deviation of the adjacent capture angles γ_{ab} and γ_{ac} from the desired value $2\pi/N$, and r_{a2} penalizes the inconsistency between the two adjacent capture angles, with $\Delta\gamma_a = |\gamma_{ab} - \gamma_{ac}|$. These terms jointly encourage an evenly spaced and stable encirclement during the rounding-up process. Note that the hard containment constraint on adjacent capture angles (Section III-A) is enforced by the success/termination condition; the angle reward in Eq. (21) is used only as smooth reward shaping to accelerate convergence.

In summary, the reward design implements the mechanism of “approach → synchronize → regulate angles → maintain

safety”, so that the team can realize geometric closure even when each agent only observes local information.

From the zero-sum game model established in Section IV-A, the reward function of the evader USV is obtained as follows:

$$R_e = - \sum_{i=1}^N R_p^i. \quad (22)$$

At the same time, in order to maintain the stability of the evader’s reward value (not to fluctuate greatly with the number of chasing agents), the average value is taken in the actual training process. The constructed reward function ensures that the Multi-Agent Reinforcement learning method can search for Nash equilibrium strategy in the pursuit and escape differential game problem.

The pursuer reward in Eq. (16) is a weighted combination of four components (goal, collision, distance, and angle), where σ_g , σ_{col} , σ_d , and σ_a control their relative contributions. Our coefficient selection follows a scale-balancing principle: each reward term is designed to have a comparable magnitude (order of 1) during typical interactions, so that no single component dominates the learning signal throughout training. In addition, safety-related penalties are intentionally weighted to be sufficiently strong so that collision avoidance is treated as a hard preference during exploration, rather than being traded off for short-term distance reduction. For clarity and reproducibility, we report the exact coefficient values used in all experiments in the experimental setup description.

To assess robustness with respect to these hyperparameters, we specify a sensitivity evaluation protocol in which σ_g and σ_{col} are varied within a small neighborhood around the reported setting (while keeping other coefficients fixed), and the resulting trends in capture success rate, collision rate, and average steps are examined under the same training budget and unseen-map testing procedure. This protocol helps verify that the proposed method does not rely on a single “fragile” choice of reward weights.

Compared with conventional actor-critic methods, the Soft Actor-Critic (SAC) paradigm optimizes an entropy-regularized objective, which maintains a stochastic policy and encourages exploration while learning. As an off-policy actor-critic method, SAC can reuse past transitions through experience replay, leading to improved sample efficiency and more stable learning in continuous action spaces. These properties are particularly beneficial for multi-USV pursuit-evasion in obstacle-rich environments, where informative interactions (e.g., near-collision and near-capture events) may be sparse and safe exploration is crucial. On the other hand, SAC/MASAC may introduce additional computational overhead (e.g., multiple critics and entropy tuning) and can be sensitive to reward scaling and hyperparameter choices; moreover, in multi-agent adversarial settings, non-stationarity may still affect training stability. In this work, we mitigate these issues via centralized training with decentralized execution, target networks (Polyak aver-

aging), prioritized experience replay, and alternating updates between pursuer and evader policies.

For clarity, we briefly summarize the underlying optimization principle of SAC/MASAC used in this work. Our method follows the maximum-entropy actor-critic objective, where the policy is trained to maximize the expected return while maintaining sufficient stochasticity via an entropy term. Accordingly, the critic is optimized by minimizing the soft Bellman residual with a target that includes both the next-step Q-value and the entropy term, while the actor is updated by minimizing the KL-equivalent objective that trades off the expected Q-value and the log-likelihood (entropy-regularized) term. The detailed derivations of these standard updates can be found in the original SAC literature [19], and our implementation adopts this principle within the proposed POSG-based multi-agent setting.

V. PROPOSED MASAC-BASED COOPERATIVE CONTAINMENT METHOD

Underlying mechanism (problem-driven view): The containment task studied in this paper is challenging because each USV only has partial observations, obstacles can invalidate naive shortest-path interception, and successful capture requires multi-agent *geometric closure* rather than single-agent pointwise pursuit. Accordingly, the proposed MASAC-based method is organized as a set of mechanisms that explicitly map local information to closure-inducing behaviors: (i) PER increases the learning frequency of rare but decisive transitions (e.g., near-capture, near-collision, and adversarial escape events), accelerating credit assignment for closure-critical decisions; (ii) the Obstacle Awareness Block (OAB) transforms raw beam ranges into obstacle-risk-aware latent features so that safety constraints are respected under partial observability; (iii) the Feature Embedding Block (FEB) converts variable-size neighbor observations into fixed-dimensional representations, enabling scalable coordination without retraining for different swarm sizes; (iv) the policy network uses temporal modeling (MSA+ITS) to compensate for partial observability by integrating history, thus stabilizing multi-step encirclement formation; and (v) the CTDE value network explicitly fuses pursuer-side aggregate information and evader-side information during training, enabling robust adversarial learning while preserving decentralized execution.

A. PRIORITY EXPERIENCE REPLAY MECHANISM

Traditional reinforcement learning methods typically employ a uniform sampling strategy in their experience replay (ER) mechanism during training. However, this approach exhibits limited performance in complex tasks, particularly due to insufficient utilization of critical experiences and under-sampled high-error transitions. To address this limitation, Prioritized Experience Replay (PER) enhances sample efficiency and learning speed by assigning sampling priorities proportional to temporal difference (TD) errors, thereby prioritizing high-error experiences. To improve sample

quality and convergence performance, this work introduces a prioritized replay buffer to optimize the experience replay mechanism in the Soft Actor-Critic (SAC) framework.

Mechanism relevance to the concerned problem: In cooperative containment with obstacles and an adaptive evader, the transitions that determine success are often sparse and highly informative (e.g., a brief opening in the formation, a near-miss collision, or a sudden evader maneuver). Uniform replay tends to under-sample such “decision points”, leading to slow acquisition of closure behaviors and unstable safety performance. PER addresses this by biasing sampling toward large TD-error transitions, which typically correspond to unexpected outcomes under partial observability; thus the agents learn faster to correct formation-breaking actions and to reinforce closure-preserving responses.

The core of the Prioritized Experience Replay buffer lies in importance-proportional sampling of experiences. In traditional experience replay, all samples are treated uniformly, whereas PER assigns priorities to each transition based on Temporal Difference (TD) errors, formulated as:

$$p_t = |\delta_t| + \varepsilon, \quad (23)$$

where $\delta_t = Q_{target} - Q(s, a)$ denotes the Temporal Difference (TD) error of sample t , calculated as the discrepancy between Q_{target} (the target Q-value described in Section III-C) and $Q(s, a)$ (the Q-value estimated by the value network for taking action a in state s). A small positive constant ε is introduced to prevent zero priorities. The sampling probability for sample t is determined by $P_t = \frac{p_t^{\alpha_{prb}}}{\sum_k p_k^{\alpha_{prb}}}$, where α_{prb} is the hyperparameter in prioritized experience replay that controls the degree of prioritization, ensuring higher sampling probability for transitions with elevated TD errors.

For each sample t , the importance sampling weight calculation formula is:

$$w_t = (N \cdot P_t)^{-\beta_{prb}}. \quad (24)$$

Within this framework, N represents the total number of samples, and β_{prb} denotes the importance sampling weights in prioritized experience replay, which regulate the degree of importance sampling correction. The hyperparameter β_{prb} is typically scheduled to linearly increase to 1 during training. In practical implementations, w_t is normalized to ensure numerical stability during training.

The core workflow of the prioritized experience replay strategy comprises four stages: priority computation, experience sampling, importance sampling weight calculation, and network updating. During priority computation, the priority of each experience transition is determined based on its Temporal Difference (TD) error and stored in the replay buffer to emphasize critical experiences in subsequent sampling. Experience sampling involves non-uniform selection of transitions according to the priority distribution, ensuring higher-frequency utilization of high-priority samples for network optimization. Importance sampling

weight calculation corrects the bias introduced by prioritized sampling through weights inversely proportional to sampling probabilities, thereby preserving training unbiasedness. Network updating leverages the sampled experiences and corresponding weights to refine both policy and value networks, while dynamically adjusting experience priorities using the latest TD errors. By iterating through these stages, prioritized experience replay significantly enhances sample utilization efficiency and training stability, enabling superior algorithmic performance in complex environments.

B. DESIGN OF OBSTACLE AWARENESS BLOCK

For collaborative target capture tasks in complex obstacle environments, this section implements the obstacle-feature encoder $f_{obs}(\cdot)$ in Section III-B as an Obstacle Awareness Block (OAB) based on hybrid feature encoding and dynamic gating mechanisms, as shown in Fig. 5. The core innovation lies in three key designs:

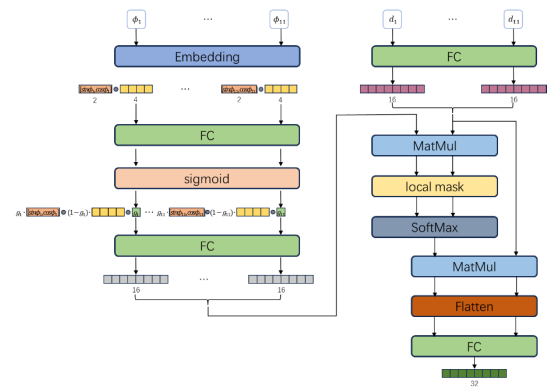


FIGURE 5. Schematic of obstacle awareness block network structure.

Mechanism relevance to the concerned problem: Obstacle avoidance is not an auxiliary objective in containment; it directly affects whether the pursuers can maintain a closed formation without creating escape corridors. Under partial observability, the policy must infer obstacle threat levels from local beam measurements and react in a way that preserves both safety and formation integrity. The OAB provides a structured inductive bias: it encodes geometric beam layout (spatial prior), learns obstacle saliency from data (semantic latent), and uses a dynamic gate to balance these two sources so the same policy can adapt to different obstacle densities and shapes.

To build a dual-stream coding system that integrates geometric priors with data-driven features, three components are designed to work in sequence. The geometric coding layer leverages sensor transmission angles (e.g., uniformly distributed from -60° to $+60^\circ$) to compute trigonometric feature vectors for each beam, constructing an 11×2 geometric encoding matrix that preserves prior knowledge of physical spatial relationships. The learnable encoding layer then generates 4-dimensional feature vectors for each of the 11 sonar beams through an embedding mechanism,

capturing data-driven environmental interaction patterns and transforming raw distance measurements into latent representations that encode obstacle saliency and spatial-temporal dependencies. Finally, the dynamic fusion gate employs a gated weighting network, consisting of fully connected layers with sigmoid activation, to automatically produce 11×1 fusion coefficients from the concatenated geometric and learned features, resulting in a 7-dimensional obstacle descriptor for each beam that combines positional priors, learned semantics, and estimated threat levels.

Localized Perception Modeling: Incorporates a band-shaped neighborhood mask into attention computation, constraining each sensor to focus only on adjacent ± 1 beam data (forming a three-beam perception window). This design emulates biological visual focus characteristics while reducing computational complexity. **Key-Value Projection:** Projects raw distance signals into 16-dimensional key vectors via linear layers, while generating query vectors from hybrid encoded features. Inter-sensor relevance scores are computed through query-key dot product operations. **Multi-Agent Parameter Sharing:** All agents share identical attention parameter matrices, ensuring swarm-wide consistency in responding to similar obstacle patterns.

Spatial Feature Aggregation: Flattens the 11×16 -dimensional attention features while preserving spatial awareness through positional encoding retention. **Nonlinear Mapping:** Extracts high-level obstacle-avoidance semantic features via two fully connected layers with ReLU activation functions, achieving progressive dimensionality reduction.

This design enhances feature representation capability through geometry-data dual-stream encoding, reduces computational complexity via localized attention mechanisms, and enables the model to autonomously balance collision warning and path smoothness through dynamic gating.

Obstacle avoidance in the proposed multi-USV pursuit-evasion framework is achieved through a combination of perception, representation learning, and safety-oriented optimization. Specifically, each USV acquires localized obstacle information via beam-sensor distance measurements, which are included in the observation and further encoded by the proposed Obstacle Awareness Block (OAB) to capture obstacle-interaction patterns. In addition, collisions with obstacles/other agents are explicitly discouraged by the collision-penalty term in Eq. (19) (weighted in Eq. (17)), and each episode terminates immediately upon collision. Together, these designs encourage collision-free trajectories while pursuing the encirclement objective under boundary constraints.

C. DESIGN OF FEATURE EMBEDDING BLOCK

To address the challenge of processing variable-length multi-agent observations for network input, this section implements the variable-length feature encoder $f_{var}(\cdot)$ in Section III-B as a Feature Embedding Block (FEB) that maps a set of homogeneous items into a fixed-dimensional representation. However, this method exhibits critical limitations: the

dimensionality of observations scales linearly with swarm size, necessitating separate policy network training for each swarm size configuration. Such vectorized inputs inherently lack adaptability to dynamic scenarios with varying swarm sizes, severely restricting model scalability and hindering deployment in large-scale swarm systems. To overcome these constraints, this work introduces a Feature Embedding Block (FEB) designed to encode variable quantities of homogeneous state features into fixed-dimensional representations.

Mechanism relevance to the concerned problem: Geometric closure depends on relative configuration (neighbors and angles), not on the absolute ordering or the exact number of pursuers. Therefore, the policy should be permutation-tolerant and size-adaptive: it must extract “who is near me” and “how the formation looks” from a variable number of neighbors, using a fixed-size representation. The FEB implements this by shared-weight item encoders (homogeneous processing) and pooling-based compression (set aggregation), allowing the same policy to generalize across different swarm sizes while preserving the key coordination cues.

The input to the Feature Embedding Block (FEB) consists of variable quantities of observational features. First, homogeneous attributes of different items are normalized to ensure the observations are on the same numerical scale. Then, each item is processed through fully connected layers with shared weights to obtain features of identical dimensions. Finally, two feature compression methods are designed: column-wise max-pooling (CMP) and column-wise average-pooling (CAP). The outputs of these methods are concatenated to generate the embedded features, as shown in Fig. 6.

For pursuer a , each item in O_a^{in} is a 5-D vector $[d_{ab}, \theta_{ab}, \theta_{ba}, \Delta d_{ab}, \gamma_{ab}]$. FEB applies shared fully connected layers to each item, followed by column-wise max-pooling (CMP) and average-pooling (CAP) across items. The pooled features are concatenated and mapped to a final 32-D embedding. For the evader, each item in O_e^{out} is 2-D $[\psi_a, d_a]$, and the shared mapping is $2 \rightarrow 32 \rightarrow 32$ with the same pooling and 32-D output.

D. POLICY NETWORK STRUCTURE DESIGN

In decision-making tasks under partially observable environments, agents' single-frame observations suffer from inherent limitations in incomplete state representation, leading to state estimation biases in models relying solely on instantaneous observations. To address this challenge, this study innovatively proposes a Masked Self-Attention (MSA) module that reconstructs the global environmental state through temporal feature fusion. To balance model accuracy with computational efficiency, we further introduce an Interval-based Temporal Sampling (ITS) strategy, which compresses long-term observation sequences using an exponentially decaying sampling function. This method preserves the integrity of critical temporal information while significantly reducing computational complexity, enabling efficient and

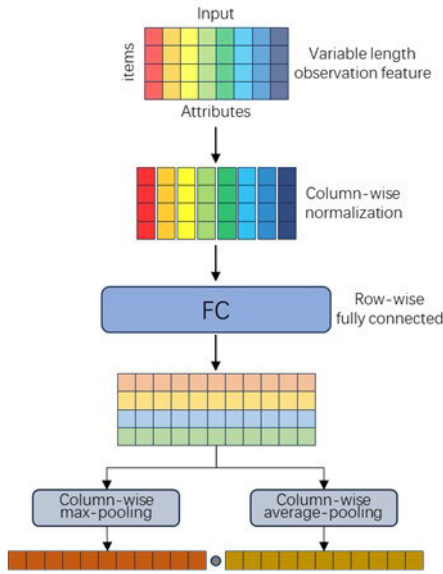


FIGURE 6. Schematic of feature embedding block network structure.

precise state estimation in dynamic partially observable scenarios.

We maintain a short history buffer of past encoded features and apply interval-based temporal sampling (ITS) to select K frames with increasing time gaps for efficiency. The sampled sequence $X \in \mathbb{R}^{K \times d}$ is fed into a transformer-style masked self-attention (MSA) module. In our implementation, $d = 32$ and the attention uses H heads. A binary mask is used to ignore padded/unavailable frames. The attention output of the current time step is used as the temporally enhanced feature for action generation.

Mechanism relevance to the concerned problem: Containment is inherently multi-step: agents must approach, coordinate timing, and regulate relative angles while reacting to an adversarial evader and obstacles. With partial observations, a single frame cannot disambiguate whether an observed opening is transient or persistent, nor whether a neighbor is lagging behind. MSA+ITS provides a compact memory mechanism: it aggregates historical cues needed for stable formation maintenance and reduces sensitivity to instantaneous sensing noise, thereby supporting the emergence of blockade-and-compress behavior instead of myopic chasing.

For the i -th pursuer USV, the policy network integrates three types of observations: variable length features o_{in}^i , invariable length features o_{out}^i , and obstacle features from beam sensors D_i . The obstacle features D_i are first processed by the OAB to generate a 32-dimensional obstacle-aware feature. Concurrently, the variable-length features o_{in}^i are encoded into a 32-dimensional embedded feature through the FEB, while the invariable length features o_{out}^i are transformed into a 32-dimensional containment feature via a single fully connected layer. These three features are concatenated into a 96-dimensional vector, which is further processed by two

fully connected layers and the Masked Self-Attention (MSA) module to incorporate temporal dependencies, resulting in a 32-dimensional temporally enhanced feature. Finally, two independent fully connected layers output the mean $\mu = [\mu_u, \mu_r]$ and standard deviation $\sigma = [\sigma_u, \sigma_r]$ of the Gaussian distribution for the action $a_i = \{u_i, r_i\}$, determining the surge velocity and yaw rate. This architecture, illustrated in Fig. 7, ensures scalable and adaptive decision-making by dynamically integrating heterogeneous observations while maintaining computational efficiency through modular feature fusion and hierarchical temporal modeling.

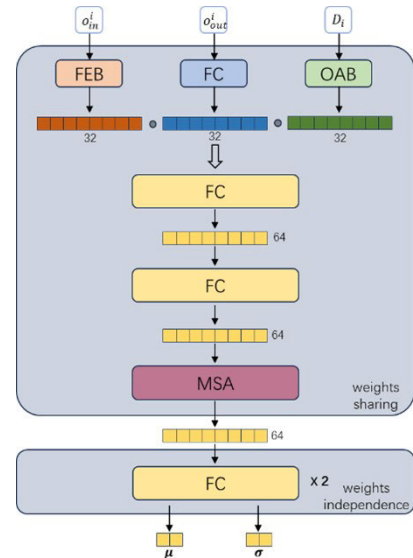


FIGURE 7. Schematic of pursuit USV policy network structure.

Unlike pursuer USVs, the evader USV possesses only variable-length observational features o_{out}^e and obstacle-sensing features from beam sensors D_e , without fixed-length observations. The obstacle-sensing features D_e are processed through the OAB to generate a 32-dimensional obstacle-aware feature, while the variable-length features o_{out}^e are encoded into a 32-dimensional embedded feature via the FEB. These two features are concatenated into a 64-dimensional vector. Subsequent processing mirrors the approach used for pursuer USVs: the fused features undergo refinement through two fully connected layers and the MSA module, yielding a temporally enhanced 32-dimensional representation. Finally, two independent fully connected layers output the mean $\mu = [\mu_u, \mu_r]$ and standard deviation $\sigma = [\sigma_u, \sigma_r]$ of the Gaussian distribution for the evader's action $a_e = \{u_e, r_e\}$, which governs surge velocity and yaw rate adjustments. This streamlined architecture, depicted in Fig. 8, ensures computational efficiency while maintaining adaptive evasion capabilities through selective feature integration and dynamic temporal reasoning.

E. VALUE NETWORK STRUCTURE DESIGN

This study employs the Centralized Training with Decentralized Execution (CTDE) framework, widely adopted in

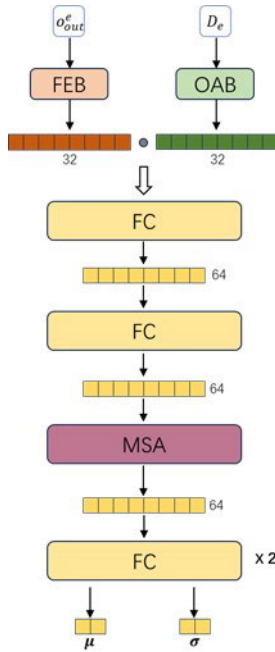


FIGURE 8. Schematic of evader USV policy network structure.

multi-agent reinforcement learning, to collaboratively optimize agents through global state information while retaining their independent decision-making capabilities. Innovatively tailored to pursuit-evasion scenarios, a dual-end feature-fused value network architecture is designed: Pursuer-end processing: A shared-weight encoder network processes observations from all agents, extracts features, and generates swarm-level representations via mean pooling; Evader-end processing: Independent feature encoding is performed based on single-agent observational data.

Mechanism relevance to the concerned problem: The pursuers must learn cooperative closure against an adaptive evader; however, decentralized execution alone makes credit assignment difficult because each agent cannot observe the full formation state. CTDE resolves this by allowing the critic to use richer (training-time) information to evaluate joint actions, while the actors remain deployable using only local observations. The proposed dual-end fusion further matches the pursuit-evasion structure: the pursuer-side pooling captures formation-level containment status, and the evader-side encoding captures adversarial escape context; their fusion produces Q-values that reflect the coupled game dynamics.

To achieve cross-role information interaction, the aggregated features from the pursuer end and the individual features from the evader end are tensor-concatenated, along with the action space information from both parties. This combined representation is then processed through two-layer independent fully connected networks for dual-end information fusion, ultimately mapping to the Q-values of each agent, as shown in Fig. 9. This architecture enables efficient adversarial strategy learning by explicitly modeling

interdependencies between cooperative pursuit and competitive evasion behaviors while maintaining computational tractability in large-scale swarm scenarios.

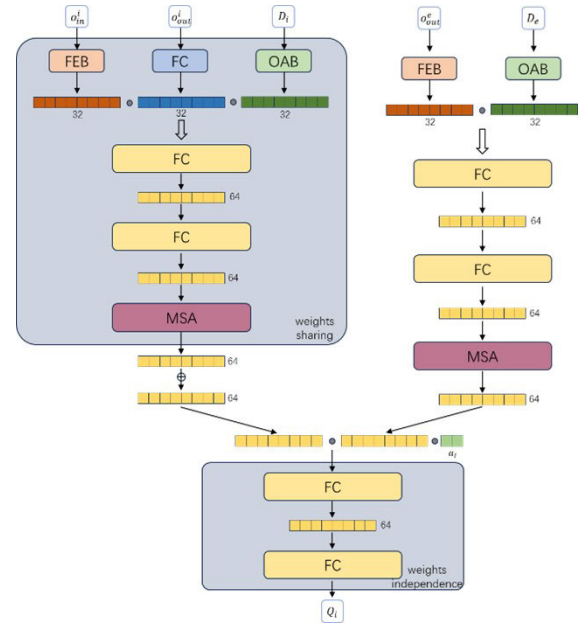


FIGURE 9. Schematic of value network structure.

F. MASAC ALGORITHM FLOW DESIGN

This study employs the Centralized Training with Decentralized Execution (CTDE) framework to achieve collaborative target capture tasks for USVs. During the training phase, a centralized controller is established to access global state information for optimizing agent policies through coordinated learning. In the execution phase, each agent independently makes action decisions based solely on its local observations, without requiring knowledge of other agents' states or actions. The framework flowchart is in Fig. 10.

Mechanism summary of the training loop: The algorithm alternates between (i) collecting partially observed interaction trajectories in obstacle-rich environments, (ii) emphasizing closure- and safety-critical transitions via PER, and (iii) updating decentralized policies under a centralized critic. This mechanism ensures that the learned policies are both deployable (local observations only) and effective for the concerned containment objective (emergent geometric closure under obstacles and adversarial evasion).

During training, the environment outputs the global state s_t , and then the reward function R and observation function Φ output the rewards r_t^i and observations o_t^i obtained by each USV in this step of training according to the state s_t . The USV strategy network outputs actions a_t^i according to the observations o_t^i , and finally the environment gives the updated global state s_{t+1} according to the joint actions $\mathbf{a}_t$ of all USVs, which means that an interactive cycle is completed. During each cycle, the intermediate state

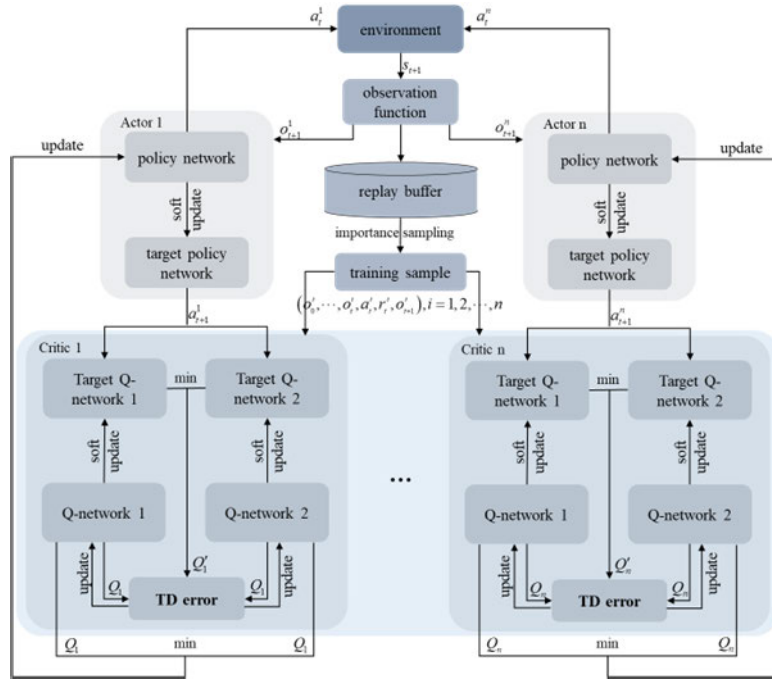


FIGURE 10. Flowchart of MASAC algorithm framework.

data is stored in the experience replay buffer in the form of $(o_0^i, \dots, o_t^i, a_t^i, r_t^i, o_{t+1}^i)$, $i = 1, 2, \dots, n$. When the collected experience reaches the specified capacity, calculate the update gradient of the policy network and value network, and update the network weight using Adam optimizer.

The pseudocode of the algorithm is shown in Table 2.

Although the proposed method is implemented with function approximation in a partially observable stochastic game (POSG), its learning objective follows the entropy-regularized (maximum-entropy) soft policy iteration principle. In particular, for a fixed joint policy, the corresponding soft Bellman backup defines the soft state–action value update, and the induced soft Bellman operator is a contraction in the tabular setting under bounded rewards. Consequently, the soft policy evaluation step converges to a unique fixed point, and the subsequent soft policy improvement step is guaranteed to be non-decreasing in the maximum-entropy objective.

In our multi-agent setting, we adopt centralized training with decentralized execution: the critic is trained using joint information to mitigate non-stationarity, while each actor optimizes a local stochastic policy conditioned on its observation. Practically, stability is further promoted by (i) target networks with Polyak averaging (soft update factor τ), (ii) entropy regularization that prevents premature collapse to deterministic and unsafe behaviors, (iii) prioritized experience replay that emphasizes high-TD-error transitions (e.g., near-collision and near-capture events), and (iv) alternating updates of pursuer and evader networks, which alleviates the non-stationarity induced by adversarial learning. These

mechanisms together provide a principled explanation for the stable learning trend observed in Fig. 11.

VI. SIMULATION EXPERIMENT AND ANALYSIS

In this section, we developed a multi-agent containment simulation environment and trained the USV policy networks using the proposed algorithm, followed by an examination of its effectiveness. Finally, comparative analyses were conducted to evaluate the algorithm’s generalizability and robustness.

A. EXPERIMENTAL ENVIRONMENT AND PARAMETER SETTING

The experimental setup in this section adopts the following environment configuration: CPU Xeon(R) Platinum 8362, GPU RTX 3090 24GB. The algorithm is implemented in Python and built on the PyTorch deep learning framework. Additional hyperparameter settings are detailed in Table 3.

An episode terminates when the pursuer USV swarm successfully contains the evader USV, collides with obstacles, or the time steps reach 500. Training commences once the experience replay buffer accumulates 2000 samples, with initial training conditions specified in Table 4.

The task is simulated in a 2D bounded workspace with boundary range $[0, 100]$. Obstacles are modeled as static circular regions within each episode; different obstacle layouts are randomly generated across episodes to represent “unknown environments.” Agents do not access a global obstacle map; obstacle perception is provided only via the forward-looking beam sensor (11 beams, 120° field-of-view,

TABLE 2. Algorithm 1: MASAC algorithm.

Step	Algorithm 1 MASAC algorithm
0	Input: multi-agent environment $\mathcal{E}$; number of agents N ; maximum episodes M ; horizon T ; discount factor γ ; soft update coefficient τ ; entropy temperature $\{\alpha^i\}_{i=1}^N$; replay buffer $\mathcal{D}$; batch size $ \mathcal{B} $; Adam learning rates. Output: trained decentralized policies $\{\pi_{\theta^i}(\cdot o^i)\}_{i=1}^N$ and centralized critics $\{Q_{\omega_1^i}, Q_{\omega_2^i}\}_{i=1}^N$.
1	Initialize centralized critic networks for each agent i : $Q_{\omega_1^i}(s, \mathbf{a})$, $Q_{\omega_2^i}(s, \mathbf{a})$ and actor network $\pi_{\theta^i}(\cdot o^i)$.
2	Copy the same parameter for each agent i : $\omega_1^{i,target} \leftarrow \omega_1^i, \omega_2^{i,target} \leftarrow \omega_2^i, \theta^{i,target} \leftarrow \theta^i$ to initialize the target networks respectively $Q_{\omega_1^{i,target}}, Q_{\omega_2^{i,target}}, \pi_{\theta^{i,target}}$.
3	Initialize experience pool $\mathcal{D}$.
4	for episode = 1 $\rightarrow M$ do
5	Get the initial global state s_0 of the environment and local observations $\{o_0^i\}_{i=1}^N$.
6	for timestep $t = 0 \rightarrow T - 1$ do
7	For each agent $i = 1, \dots, N$, select an action $a_t^i \sim \pi_{\theta^i}(\cdot o_t^i)$.
8	Execute joint action $\mathbf{a}_t = (a_t^1, \dots, a_t^N)$ and gain rewards $\{r_t^i\}_{i=1}^N$, next global state s_{t+1} and next observations $\{o_{t+1}^i\}_{i=1}^N$.
9	Store the sample $(s_t, \{o_t^i\}_{i=1}^N, \mathbf{a}_t, \{r_t^i\}_{i=1}^N, s_{t+1}, \{o_{t+1}^i\}_{i=1}^N)$ in the experience pool $\mathcal{D}$.
10	Random sampling of batch $\mathcal{B}$ samples from the experience pool $(s_t, \{o_t^i\}, \mathbf{a}_t, \{r_t^i\}, s_{t+1}, \{o_{t+1}^i\})$.
11	For each agent i , calculate the target Q-value: sample target actions $\tilde{a}_{t+1}^i \sim \pi_{\theta^{i,target}}(\cdot o_{t+1}^i)$ and construct $\tilde{\mathbf{a}}_{t+1} = (\tilde{a}_{t+1}^1, \dots, \tilde{a}_{t+1}^N)$.
	$Q_{target}^i = r_t^i + \gamma \left(\min_{j=1,2} Q_{\omega_j^{i,target}}(s_{t+1}, \tilde{\mathbf{a}}_{t+1}) - \alpha^i \log \pi_{\theta^{i,target}}(\tilde{a}_{t+1}^i o_{t+1}^i) \right).$
12	The two Q-value networks of each agent i are updated according to the following loss function (for $j = 1, 2$): $\mathcal{L}_{Q_j^i} = \frac{1}{ \mathcal{B} } \sum_{\mathcal{B}} \left(Q_{\omega_j^i}(s_t, \mathbf{a}_t) - Q_{target}^i \right)^2.$
13	For each agent i , actions are sampled using the reparameterization trick and the current policy network is updated with the following loss function: sample current actions $\tilde{a}_t^i \sim \pi_{\theta^i}(\cdot o_t^i)$ and form $\tilde{\mathbf{a}}_t = (\tilde{a}_t^1, \dots, \tilde{a}_t^N)$. $\mathcal{L}_{\pi}^i = \frac{1}{ \mathcal{B} } \sum_{\mathcal{B}} \left(\alpha^i \log \pi_{\theta^i}(\tilde{a}_t^i o_t^i) - \min_{j=1,2} Q_{\omega_j^i}(s_t, \tilde{\mathbf{a}}_t) \right).$
14	For each agent to update the target network: $\theta^{i,target} \leftarrow \tau \theta^i + (1 - \tau) \theta^{i,target}, \omega_j^{i,target} \leftarrow \tau \omega_j^i + (1 - \tau) \omega_j^{i,target}, j = 1, 2$.
15	end for
16	end for

TABLE 3. Simulation hyperparameter setting.

Hyperparameter	Value
Discount factor γ	0.99
Critic network learning rate	0.00005
Actor network learning rate	0.00001
Soft update factor τ	0.005
Batch size	64
Buffer size	200000
Number of episode rounds	5000

20 m range). In the current experiments, the pursuer team consists of three homogeneous USVs ($N = 3$) as a proof-

of-concept configuration; scalability to larger/heterogeneous swarms is left as future work.

TABLE 4. Initial conditions of simulation.

Parameter	Value
Initial position of pursuer USV cluster	(0, 0), (0, 10), (0, 20)
Initial heading angle ψ of pursuer USVs	$\pi/4$
Maximum surge-direction linear velocity $u_{\max}$ of pursuer USVs	10
Maximum yaw angular velocity $r_{\max}$ of pursuer USVs	1
Initial position of evader USV	random
Initial heading angle ψ of evader USV	$\pi/4$
Maximum surge-direction linear velocity $u_{\max}$ of evader USVs	6
Maximum yaw angular velocity $r_{\max}$ of evader USVs	1.2
Radius of USV	1
Boundary range	[0, 100]

B. REPRODUCIBILITY AND CODE/MODEL AVAILABILITY

Reproducibility is an important requirement for multi-agent deep reinforcement learning methods. To facilitate replication of our results, we provide in this paper a detailed description of the simulation environment, termination conditions, state/action definitions, reward design (Eqs. (19)–(21)), training protocol, and key hyperparameters (Table 3–Table 4), as well as the testing procedure on unseen environments (Fig. 14–Fig. 15).

C. SIMULATION RESULTS AND ANALYSIS

In the training phase, the value network and the action network are trained at the same time. Before 2000 rounds, the network of pursuers and evaders is alternately trained by using the priority experience playback strategy. After 2000 rounds, it is judged whether the pursuer cluster has completed the round up task. If it is completed, the evader network is trained, otherwise the pursuer network is trained. Fig. 11 shows the total return of all agents on both sides during the training process. It can be seen that after about 3000 rounds, the total return of all agents on both sides of the pursuit and evasion tends to be stable.

To discuss the reasons behind the convergence trend in Fig. 11, we relate the learning dynamics to the shaped rewards and the adopted training scheme. The pursuer-side distance reward in Eq. (20) is dense and provides informative gradients in most states, enabling the policy to rapidly learn a coarse distance-closing behavior, which yields a fast increase of returns in early training. Meanwhile, the exponential term in Eq. (20) introduces a team-level synchronization pressure by penalizing agents whose remaining capture distance is larger than the team average, thereby reducing “one-agent-lagging” cases and gradually stabilizing the group behavior. After this stage, further performance improvement mainly depends on learning the higher-order geometric coordination required by the angle reward in Eq. (21) (i.e., maintaining appropriate adjacent capture angles), as well as satisfying the safety constraints induced by the collision penalty in Eq. (19); these objectives are harder and improve more slowly, so the overall return curve tends to plateau after sufficient training (around 3000 episodes). In addition,

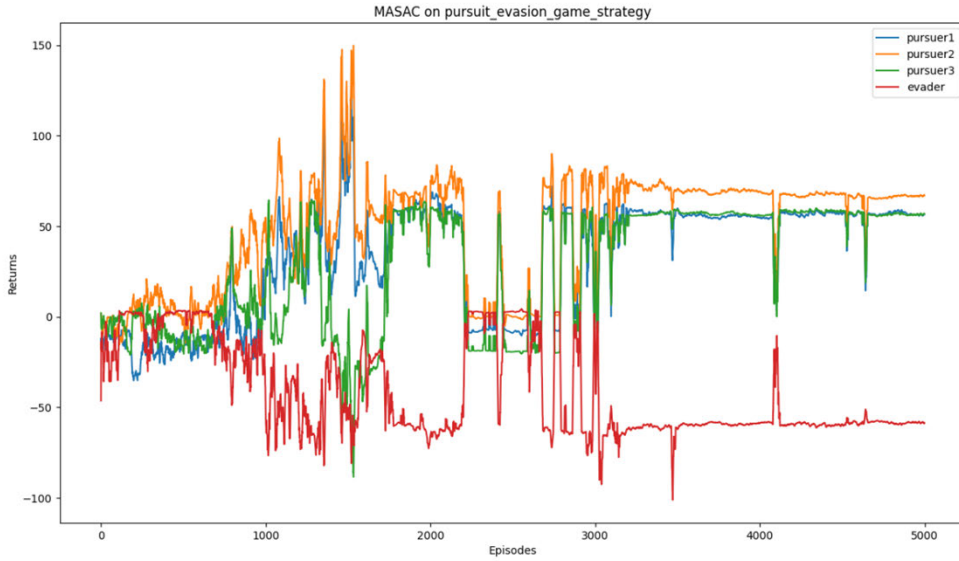


FIGURE 11. Reward curves of MASAC algorithm.

alternating updates of pursuer and evader networks alleviate the non-stationarity of the zero-sum game in Eq. (22), and prioritized experience replay emphasizes high-TD-error transitions (e.g., near-capture and near-collision events), which accelerates early learning and makes later updates less volatile.

Fig. 12 shows the trajectory of USV pursuing and escaping both sides in the 2500 round. It can be seen that although the pursuer failed to successfully round up the evader, he has learned how to approach the evader, and the evader has also learned to use obstacles to get rid of the chase. The change of the path shows that with the increase of the number of training rounds, the pursuer USV group gradually learned how to round up the evader as soon as possible under the condition of meeting the collision avoidance and boundary constraints. Fig. 13 shows the trajectory of USV pursuing and fleeing both sides under the optimal strategy after 5000 rounds. The pursuer has an advantage over the evader, so that the pursuer can successfully round up the evader. The change of the path shows that with the increase of the number of training rounds, the pursuer USV group gradually learned how to round up the evader as soon as possible under the condition of meeting the collision avoidance and boundary constraints. It can be seen that with the training, the success rate of the pursuer's rounding up gradually increases, and the evader's strategy makes more use of obstacles and mobility to achieve more flexible escape. These behaviors show that MASAC gradually converges to a strategy with practical significance.

Discussion on the reasons behind the trajectory evolution (Fig. 12–Fig. 13). The mid-training behaviors in Fig. 12 (episode 2500) can be explained by the staged effect of the shaped rewards. The distance reward $r_{distance}^a$ in Eq. (20) dominates early learning and drives each

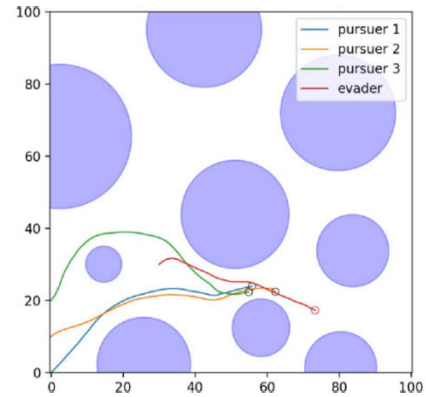


FIGURE 12. The trajectory of USV pursuing and escaping both sides in the 2500 round.

pursuer to reduce its distance to the evader, so the pursuers first acquire a consistent “approach” strategy. Moreover, the synchronization-related exponential term in Eq. (20) penalizes pursuers whose remaining capture distance d_a is larger than the team average $\bar{d}$, which encourages lagging agents to choose shorter routes and speeds up the formation of group-level synchronized pursuit. However, successful containment requires not only getting close but also forming a stable encirclement geometry, which is explicitly regulated by the angle reward r_{angle}^a in Eq. (21). Since r_{angle}^a depends on the relative configuration among multiple pursuers (adjacent angles γ_{ab}, γ_{ac} and their consistency $\Delta\gamma_a$), it represents a higher-order coordination objective; therefore, it typically emerges later than distance-closing, explaining why the pursuers can approach but may still fail to complete encirclement at 2500 episodes. The evader's obstacle-assisted escape in Fig. 12 is also consistent with the zero-sum

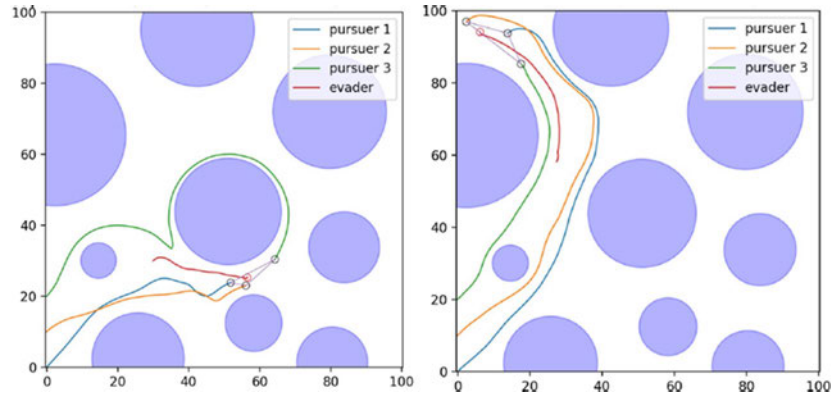


FIGURE 13. Trajectory of USV pursuit-evasion under the optimal strategy: (a) scenario 1; (b) scenario 2.

coupling in Eq. (22). By maximizing $R_e = -\sum_{i=1}^N R_p^i$ (implemented as an averaged version in training), the evader is incentivized to reduce pursuer returns. In practice, moving toward obstacles/boundaries can increase pursuers' collision risks (thus triggering the penalty in Eq. (19)) and force different pursuers into different detours, which increases the dispersion of $\{d_i\}$ in Eq. (20) and breaks the geometric conditions encouraged by Eq. (21). Therefore, the learned evasion behavior naturally exploits obstacles to disrupt coordination rather than simply maximizing instantaneous distance. After sufficient training (Fig. 13, episode 5000), the pursuers succeed because the policy has learned to optimize the complete mechanism “approach $\rightarrow$ synchronize $\rightarrow$ regulate angles $\rightarrow$ maintain safety”. Specifically, Eq. (20) drives fast and synchronized closing, Eq. (21) regularizes the formation toward equal phase division $2\pi/N$ and balanced adjacent angles, and Eq. (19) discourages unsafe shortcuts. As a result, the final policy exhibits coordinated containment under obstacle and boundary constraints, which is reflected in the successful rounding-up trajectory.

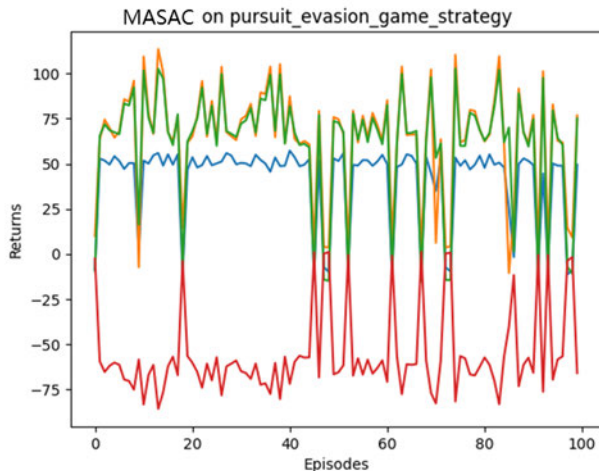


FIGURE 14. Return curves in testing.

D. COMPARATIVE ANALYSIS WITH RELATED METHODS

To further evaluate the performance superiority of the proposed algorithm, we conducted comparative experiments with three updated and representative multi-agent reinforcement learning methods: MADDPG (centralized training with decentralized execution), MAPPO (multi-agent PPO), and MATD3 (multi-agent TD3). All baseline methods were trained in the same simulation environment with identical reward functions and constraints. We evaluated the methods based on Capture Success Rate (percentage of successful containment), Average Steps (time efficiency in successful episodes), Collision Rate (percentage of episodes ending in obstacle/boundary collisions), and Average Return.

Overall, the proposed MASAC-based framework achieves a high capture success rate while maintaining a low collision risk in obstacle-rich environments. The entropy-regularized objective in MASAC encourages sufficient exploration and helps avoid premature convergence to unsafe or suboptimal behaviors (e.g., getting trapped near obstacles or local minima). Moreover, the off-policy training with experience replay improves sample efficiency by repeatedly leveraging informative transitions, including rare but safety-critical boundary-interaction experiences. These observations indicate that our method effectively balances aggressive pursuit and safety constraints, leading to stable containment performance in the considered USV pursuit-evasion scenarios.

The aforementioned simulation experiments validate the improved MASAC algorithm's local path planning capability in specific unknown environments. However, unknown environments inherently exhibit structural diversity, where environmental parameters such as obstacle distributions and aquatic boundaries may vary significantly across different marine regions. If the algorithm is solely optimized for a specific environmental configuration, its robustness in heterogeneous unknown scenarios cannot be guaranteed. To verify whether the local path planning algorithm truly possesses environment-agnostic autonomous decision-making capability, cross-scenario generalization tests were conducted: the trained policy model was directly transferred

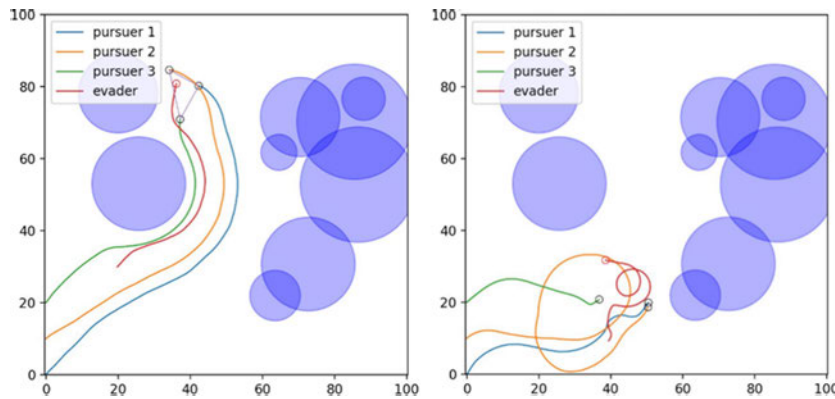


FIGURE 15. Successful and failed capture cases in testing: (a) successful capture; (b) failed capture.

to novel map environments excluded from training and evaluated over 100 test episodes. The 100 unseen-map tests are conducted on randomly generated obstacle layouts with varying obstacle densities and spatial distributions, while keeping sensing settings unchanged. Fig. 14 illustrates the total reward values for both pursuers and evaders across these tests, while Fig. 15 showcases representative successful and failed cases. As shown in Fig. 14, the algorithm achieves an 85% capture success rate in unseen environments, demonstrating strong generalization capabilities.

Reason analysis for generalization results (Fig. 14–Fig. 15). The 85% capture success rate in unseen maps suggests that the learned pursuer policies capture transferable interaction patterns rather than memorizing a single obstacle layout. Successful cases usually occur when the environment still allows the pursuers to (i) maintain continuous pressure on the evader with feasible collision-free paths and (ii) re-form an encirclement after obstacle-induced splitting. In these cases, even if some agents detour, the team can recover synchronized closing (Eq. (20)) and restore the angular configuration encouraged by Eq. (21). In contrast, the failure cases in Fig. 15 typically arise when obstacles create narrow passages or multiple branching escape corridors that amplify desynchronization: different pursuers are forced into substantially different detours or must significantly reduce speed for safe navigation, which increases the dispersion of remaining capture distances (affecting $\bar{d}$ and $\sigma(d)$ in Eq. (20)) and delays the satisfaction of the encirclement angles required by Eq. (21). The evader, whose objective is coupled through Eq. (22), can exploit these “formation break” windows by steering toward such topology-induced choke points to maximize pursuer coordination errors and collision risks. This explains why the method generalizes well in most cases but still fails in a subset of maps with strong topology-induced desynchronization.

To further strengthen the empirical validation of the proposed framework, we will extend the experimental study along two complementary directions. First, we will conduct a comprehensive benchmark evaluation against representative

multi-agent reinforcement learning baselines (e.g., MADDPG, MAPPO, and other commonly used variants) as well as classical rule-based pursuit–evasion strategies under a unified protocol, including standardized training budgets, comparable network capacities, and multi-seed reporting (mean $\pm$ std) for a fair and reproducible comparison. Second, we will perform systematic ablation and robustness analyses to quantify the contribution of key design choices and assess generalization beyond the nominal settings, such as removing the obstacle-aware beam-sensor input and/or disabling the alternating update schedule, and testing under randomized initial conditions, varying obstacle densities, and additional unseen map configurations.

VII. CONCLUSION

This paper studies multi-USV pursuit–evasion in unknown multi-obstacle environments under partial observability. We formulate the task as a POSG and propose an enhanced MASAC-based framework integrating hybrid observations, hierarchical rewards, dynamic feature compression (FEB/OAB), and temporal fusion via masked self-attention with interval-based temporal sampling. Simulation results show stable convergence and an 85% capture success rate on 100 unseen-map test episodes.

Limitations: (1) We adopt a simplified 3-DOF planar kinematic model and do not incorporate hydrodynamic disturbances (currents/wind/waves), which may cause a sim-to-real gap. (2) Obstacles are modeled as static circular regions within an episode, and the game is formulated as a multi-pursuer single-evader zero-sum capture task. (3) The current experiments validate a small homogeneous pursuer group (three USVs), and scalability to larger/heterogeneous swarms is not yet verified. (4) Additional baseline comparisons and ablation studies are not included in the current version and are planned in future work.

Future work will focus on: (1) extending to large-scale and heterogeneous swarms; (2) incorporating hydrodynamic and actuator dynamics (including latency/saturation); and (3) expanding to multi-evader and non-zero-sum objectives.

REFERENCES

- [1] R. Isaacs, *Differential Games: A Mathematical Theory With Applications To Warfare and Pursuit, Optimization and Control*. Hoboken, NJ, USA: Wiley, 1965.
- [2] I. E. Weintraub, M. Pachter, and E. Garcia, "An introduction to pursuit-evasion differential games," in *Proc. Amer. Control Conf. (ACC)*, Jul. 2020, pp. 1049–1066.
- [3] E. Garcia, Z. E. Fuchs, D. Milutinovic, D. W. Casbeer, and M. Pachter, "A geometric approach for the cooperative two-pursuer one-evader differential game," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 15209–15214, Jul. 2017.
- [4] W. Sun, P. Tsiotras, and A. J. Yezzi, "Multiplayer pursuit-evasion games in three-dimensional flow fields," *Dyn. Games Appl.*, vol. 9, no. 4, pp. 1188–1207, Dec. 2019.
- [5] L. Cao, Y. Cen, T. Li, H. Liang, and Y. Pan, "Optimal fixed-time control for human-in-the-loop multiagent systems with actuator faults," *IEEE Trans. Cybern.*, 2025.
- [6] L. Cao, Y. Qin, Y. Pan, and H. Liang, "Prescribed performance-based optimal formation control for USVs with position constraints and yaw angle time-varying partial constraints," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 3, pp. 4109–4121, Mar. 2025.
- [7] Q. Wang, K. Wu, J. Ye, Y. Wu, and L. Xue, "Apollonius partitions based pursuit-evasion game strategies by Q-learning approach," in *Proc. 41st Chin. Control Conf. (CCC)*, 2022, pp. 4843–4848.
- [8] Y. Fan, Z. Sun, and G. Wang, "A novel intelligent collision avoidance algorithm based on deep reinforcement learning approach for USV," *Ocean Eng.*, vol. 287, Nov. 2023, Art. no. 115649.
- [9] Y. Wang, L. Dong, and C. Sun, "Cooperative control for multi-player pursuit-evasion games with reinforcement learning," *Neurocomputing*, vol. 412, pp. 101–114, Oct. 2020.
- [10] L. Yuan, Z. Zhang, L. Li, C. Guan, and Y. Yu, "A survey of progress on cooperative multi-agent reinforcement learning in open environment," 2023, *arXiv:2312.01058*.
- [11] S. Gronauer and K. Diepold, "Multi-agent deep reinforcement learning: A survey," *Artif. Intell. Rev.*, vol. 55, no. 2, pp. 895–943, 2021.
- [12] P. Ladosz, L. Weng, M. Kim, and H. Oh, "Exploration in deep reinforcement learning: A survey," *Inf. Fusion*, vol. 85, pp. 1–22, Sep. 2022.
- [13] R. Zhang, Q. Zong, X. Zhang, L. Dou, and B. Tian, "Game of drones: Multi-UAV pursuit-evasion game with online motion planning by deep reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 10, pp. 7900–7909, Oct. 2023.
- [14] S. Li, Y. Wu, X. Cui, H. Dong, F. Fang, and S. Russell, "Robust multi-agent reinforcement learning via minimax deep deterministic policy gradient," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4213–4220.
- [15] X. Qu, W. Gan, D. Song, and L. Zhou, "Pursuit-evasion game strategy of USV based on deep reinforcement learning in complex multi-obstacle environment," in *Proc. Ocean Eng.*, vol. 273, 2023.
- [16] X. Yang, M. Lou, J. Hu, H. Ye, Z. Zhu, H. Shen, Z. Xiang, and B. Zhang, "A human-like collision avoidance method for USVs based on deep reinforcement learning and velocity obstacle," in *Proc. Expert Syst. Appl.*, vol. 254, 2024.
- [17] Y. Zhang, "Global path planning algorithm for multi-task amphibious unmanned surface vehicles based on double DQN," *Ocean Eng.*, 2022, Art. no. 112809.
- [18] M. Chen, "Distributed pursuit-evasion game of finite-perception USV swarm based on multi-agent proximal policy optimization," *IEEE Trans. Syst., Man, Cybern.*, 2024, Art. no. 3429467.
- [19] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.

JIAN LIU received the Ph.D. degree. He is currently a Professor. His teaching covers courses, such as operations research and system simulation. He has published more than 20 academic articles and authored two specialized textbooks. His research interests include operations optimization, system modeling and simulation, among others.

XING SHEN received the B.S. degree in mathematics from Heilongjiang University, Harbin, China, in 2021, and the M.S. degree from Qingdao University, Qingdao, China, in 2025. His research interests include swarm intelligent games, differential games, and system modeling and simulation.

HONGWEI GAO received the B.S. degree in mathematics from Jilin University, Changchun, China, in 1988, the M.S. degree from the Ocean University of China, Qingdao, China, in 1991, and the Ph.D. degree in mathematical sciences from Saint Petersburg State University, St. Petersburg, Russia, in 1997.

He is currently a Professor with Qingdao University. His research interests include swarm intelligence games, differential games, and network games.

• • •